

Ultra-Low Interactive Latency with Frameless Render-Display Integration

Evan Jonson*, Aaron Fulmer*, Hyoungsik Nam⁺, Hee-Jin Choi^o, Benjamin Watson*

*Visual Experience Lab, North Carolina State Univ., Raleigh NC USA

⁺Dept. Information Display, Kyung Hee Univ., Seoul, South Korea

^oDept. Physics and Astronomy, Sejong Univ., Seoul, South Korea

Abstract

Display applications are increasingly interactive, driving reductions in display latency. Yet display latency is only one component of total interactive click-to-photon latency, the delay users perceive. This interactive latency also includes frame buffering, GPU-display synchronization, and display refresh. This paper presents a working prototype that tightly integrates render and display, eliminating frame and synchronization delays, mitigating refresh delays, and reducing interactive latency below 2ms.

Author Keywords

Latency; frames; synchronization; refresh; progressive scan; ray tracing

1. Frames Cause Latency And May Not Be Needed

Most displays are used in interactive settings, in which a strong predictor of usability and performance is “click-to-photon” latency: the delay between user input and the resulting change on the display. Gaming displays have reduced click-to-photon latency by increasing refresh rates and mitigating other sources of latency introduced by the display itself. Yet display latency is only one part of total interactive, click-to-photon latency. To achieve further latency reductions, we are investigating tighter integration of displays with GPU renderers.

A significant source of latency has remained largely unchanged for more than 50 years: frames and their buffers. Every pixel in a frame represents the same time, meaning that by the time each frame is completely described in the frame buffer, it is already one frame time old (frame times vary depending on content complexity, but 60Hz frame rates are common, introducing 16.67ms of frame time latency). Additional delays are then caused by GPU/display synchronization (e.g. VSYNC or GSYNC) to avoid tearing, and by the display scanout. Until recently, frame buffers were essential, because rasterized rendering creates images triangle by triangle (in object order [1]), meaning that no pixel is ready for display until the last triangle is rendered. But today the need for frames and buffers is beginning to decline: with GPU support [2], many recent AAA games include ray tracing [3], and ray tracing creates images pixel by pixel (in image order [1]), meaning that pixels can feasibly be sent immediately to the display.

2. A Frameless Render-Display Architecture

This paper investigates an image-order, integrated render-display architecture designed to minimize interactive, click-to-photon latency by eliminating frames, buffers, synchronization and sequential display updates. After reviewing related work, we describe our working prototype of this architecture, built with low-cost, off-the-shelf components. We then evaluate this prototype, comparing it to a standard framed system using buffers, synchronization and a sequential progressive scan. Our prototype’s click-to-photon latency is an order of magnitude lower than the standard system’s (1.6ms vs. 32.7ms). In static moments, our prototype’s imagery is com-

parable to the standard system’s. During rapid motion, edges in our prototype’s images are less well-defined than in the standard system’s, but they are more up-to-date and more accurately depict current application state. We close by discussing the limitations of our work, the potential for translating this research architecture to industry, and avenues for future research.

3. Low Latency Needs and Frameless Systems

How low must latency go to meet user and player needs? For latency-critical applications, researchers have not yet found a floor. In gaming, Spjut et al. [4] showed that shooting and tracking performance continued improving as latency was lowered to 24ms. They did not test lower latencies, so this may not have been a floor. In touch interfaces such as phones and tablets, Jota et al. [5] reduced latency to only 1ms, and still could not conclude that a performance floor had been reached. In VR, Ellis et al. [6] found that users could perceive latency as it was lowered to 27ms. Again, this may not have been a floor. Little research addresses the relative value of latency and visual quality, but a survey of esports players [7] indicates that most prefer to turn VSYNC off, accepting the resulting drop in visual quality due to frame tearing to achieve lower latency by eliminating synchronization delay.

Given the clear harm to performance and usability of low latencies approaching 25ms, the delays of today’s framed render-display systems loom large. For example, a GPU rendering at a 60Hz frame rate adds latency well over 16ms, known to be harmful for touch interfaces, and possibly for gaming and VR as well (recall moreover that frame latency is only one component of total click-to-photon latency). Yet work studying frameless render-display systems is limited. Bishop et al. [8] first identified the problem, and simulated a simple frameless renderer offline. Dayal et al. [9] pushed this work toward practicality by making the renderer adaptive, showing that their renderer more accurately depicted current application state than a framed renderer. Friston et al. [10] created the first interactive frameless renderer, building a ray tracer on an FPGA, which fed frameless pixels just in time to a display that used a standard sequential, progressive scan. They achieved latencies under 1ms, not counting display delay, which in their system could be over 13ms. Finally, Fulmer et al. [11] built an interactive display scanning simulator in software, capable of refreshing pixels in any order, including per-pixel stochastic. None of these projects architected systems integrating both render and display, crucial for realizing ultra-low interactive, click-to-photon latencies.

4. Our Integrated Render-Display Prototype

We implement our frameless, integrated render-display prototype using a Raspberry Pi Pico 2 RP2350 and a Waveshare HUB75 RGB LED matrix panel with 64x64 pixels, 2mm pitch and a 1/32 scan rate. The Pico 2 features two ARM Cortex-M33 cores that can operate at clock speeds up to 150MHz, as well as a parallel, deterministic programmable I/O subsystem. We use the Pico to run our appli-

Table 1. NVIDIA LDAT Click-to-Photon Latency Measurements

	standard: framed, synchronized, 60Hz						prototype: frameless, integrated					
scan	progressive			stochastic			progressive			stochastic		
LDAT placement	top left	center	bottom right	top left	center	bottom right	top left	center	bottom right	top left	center	bottom right
avg ms	26.8	32.7	41.6	24.5	26.5	29.9	8.1	8.2	9.1	1.8	1.6	2.0
stdev	4.8	5.0	4.9	4.3	4.9	4.7	4.9	5.2	5.2	0.8	0.8	1.1
min ms	18.1	24.3	33.1	17.6	17.6	21.8	0.7	0.8	0.8	0.7	0.6	0.7
max ms	35.0	40.9	49.4	33.6	34.5	38.1	17.5	17.4	17.4	4.7	3.8	4.8
samples	100	100	100	100	100	100	100	100	100	100	100	100

cation, render it, and drive the display. Our application is a simple, completely playable version of Tetris. Our image-order renderer is not a full 3D ray tracer, rather it uses simple 2D pixel-object intersection tests without shading. The Pico runs quickly enough to sustain scanout at rates sufficient to simulate a fully addressable, random-access display.

We use a display with the HUB75 protocol because it is line-addressable and can operate at extremely high refresh rates. This enables us to simulate a fully random access, per-pixel addressable display, which is necessary for the tightest render-display integration. The Waveshare display requires each update to include two color lines. To simulate a single-pixel update, we write a single newly rendered pixel into a buffer containing the current image, then immediately scan out the entire line that contains it, with the other line in the pair blacked out. (Note that this buffer is not a frame buffer: its pixels can represent different times, it is only partially updated and partially scanned out, and is only necessary due to our display's refresh and addressing constraints). By doing this rapidly (without PWM color expansion, HUB75 panels can achieve 3-bit color at refresh rates over 12KHz), we can simulate the effect of true random-access pixel writes typical for interactive refresh rates, such as 60Hz.

We integrate rendering and display on this interactive simulation hardware platform pixel-by-pixel (in image order). We first sample input and application state, then render the pixel's color according to this state, then send this color immediately to the display. Note that display of this pixel does not wait for other pixels to be rendered (as in a frame buffer), for any buffer synchronization (as in VSYNC), nor for the pixel to be reached in display refresh (as in progressive scan). Note further that the render-display system is now frameless: every succeeding pixel represents a different and more up-to-date moment, with input and state sampled just in time. By eliminating delays between input and display, we substantially reduce click-to-photon latency compared to standard, framed render-display pipelines.

The order in which pixels need to be rendered and displayed is now largely arbitrary, aside from display refresh constraints. In this short paper, we focus on complete scans that visit every pixel once before any pixel is revisited.

5. Latency and Image Quality Comparison

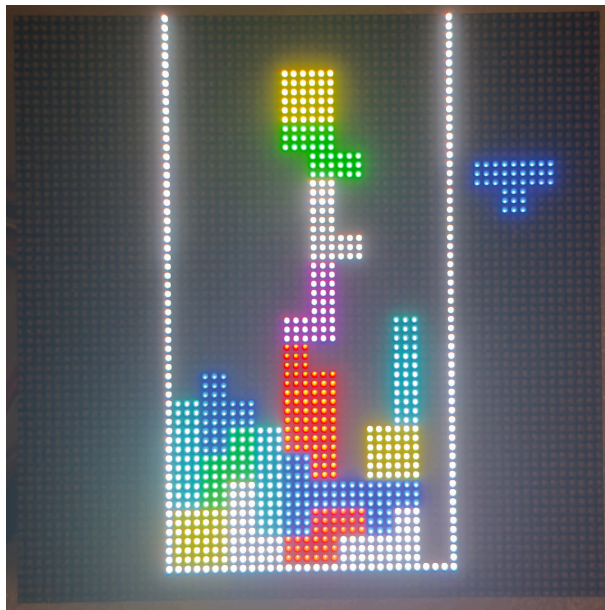
To evaluate our prototype, we compare two render-display architectures implemented on the same hardware platform: our new frameless, integrated architecture described above; and a standard framed, modular architecture. The standard architecture first samples input and application state, then using this state for every pixel in the frame it renders each pixel into the back buffer, sig-

nals the frame is ready and waits for the VSYNC buffer swap before iterating again. On a separate CPU core, the display driver swaps the buffers if the next frame is ready, scans the front buffer onto the display line by line, and then repeats this refresh process. We throttle the refresh to maintain a 60Hz rate (16.67ms/frame or 260.42μs/scanline). To test the value of our integrated architecture under realistic rendering loads, we throttle rendering speed on both architectures to the widely adopted industry standard: 60Hz frame rate (16.67ms/frame or 4.07μs/pixel).

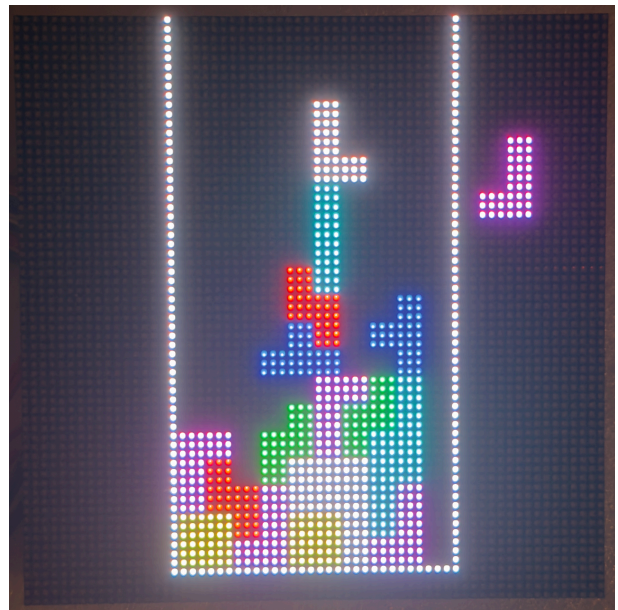
For each of these architectures, we evaluated both progressive and stochastic scans. For the standard framed architecture, the driver performs a progressive scan by sending the front buffer's lines to the display one by one, sequentially from top to bottom (the Waveshare's paired line is black). The driver performs a stochastic scan by sending the same lines in a random order fixed at initialization, mitigating refresh delays at lower regions of the display. The frameless architecture prototype performs a progressive scan by rendering and displaying lines top-to-bottom. Within each line, it renders and displays pixels left-to-right. For stochastic scan, the prototype renders and displays pixels in a random order fixed at initialization, mitigating refresh delays not only top-to-bottom, but also left-to-right.

To measure click-to-photon latency across these architectures and scans, we used NVIDIA's Display Latency and Display Analysis Tool (LDAT) [12], a discrete hardware analyzer that uses mouse input and a luminance sensor to accurately measure click-to-photon latency. We ran the LDAT software on a Windows 11 machine. We captured the mouse clicks it generated with a custom Python script and sent them to the Pico 2 serially over a USB connection, which briefly lit the display and triggered the LDAT luminance sensor. To capture latency effects resulting from render and scan order, we placed the LDAT sensor at three locations on the display: upper left, center, and lower right.

Table 1 shows the LDAT results. Our frameless, integrated prototype lowers click-to-photon latencies by an order of magnitude when rendering and displaying pixels in a stochastic order. Its average latencies are 2ms or lower, while the standard architecture's average latencies are 24.5ms or higher. Part of this improvement is due to the stochastic scan: for both the standard and frameless architectures, stochastic ordering dropped latency by 6ms on average by increasing the frequency with which each display region is updated. The remainder of our prototype's latency improvement comes from eliminating frame and synchronization delays. Note the ≈ 1 ms latency standard deviation with the prototype's stochastic scan: many users and gamers find variation in frame and refresh time ("judder") disturbing [13]. Finally, our results aptly illustrate



(a) Standard, framed architecture



(b) Prototype, frameless architecture

Figure 1. Two static moments from the game Tetris. The images in both architectures have similar visual quality. Image (a) was captured with shutter speed 1/35s, ISO 500; image (b) with shutter speed 1/33s, ISO 400.

the relationship between latency and display position in standard, progressively scanned, display architectures: latencies on the bottom right are $\approx 15\text{ms}$ higher than on the top — we are not aware of any user- or player-based justification for this design. Our prototype architecture does not relate usability to display position in this way.

Figures 1 and 2 show the visual quality of the images produced by the standard framed and prototype frameless architectures. We captured these images with an iPhone 17 Pro Max, with an exposure bias value of -0.3. Recall that these are images of the interactive game Tetris. In static moments (Figures 1a and 1b), both the standard and our prototype architecture have similar visual quality. During high-velocity motion, the captured quality of the images produced by the standard architecture are unchanged — but they are out-of-date due to latency, and do not accurately depict current game state. On the other hand, the captured quality of the images produced by our frameless architecture during high-velocity motion (Figure 2) declines, with pixels near edges showing different moments in time — more accurately depicting current game state. As noted earlier, most competitive gamers accept visual artifacts like screen tearing to reduce latency [7].

6. Discussion and Future Work

This paper proposes and interactively prototypes a frameless, integrated render-display architecture, designed to lower click-to-photon latency. While the results are promising, readers should be aware of several limitations in this work. First, our simulator was built using a largely sequential and relatively slow Pico 2 processor, displaying simple content (Tetris). Fully realized systems using our proposed architecture are likely to be implemented on highly parallelized GPUs, with correspondingly complex content (e.g., a AAA game). With this more advanced hardware and content, the advantage in click-to-photon latency for our frameless, integrated render-display architecture may not be the same as the advantage

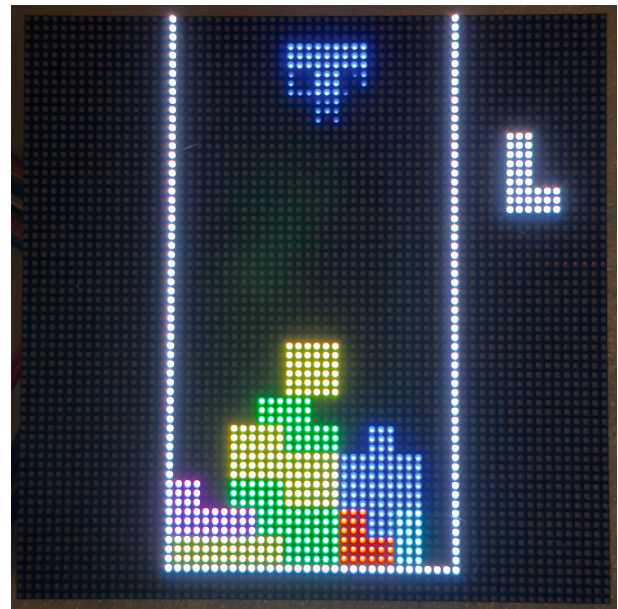


Figure 2. A moment in motion from the game Tetris, as depicted by our prototype, frameless architecture. Moving edges are not clear because pixels depict different moments, approximating game state more closely than images from the standard, framed architecture. This image was captured with shutter speed 1/24s, ISO 1000.

we found here. Moreover, most eventual applications will use much higher resolution displays than 64x64. This will add to rendering demand and affect latency, as well as changing visual quality judgments. Note also that our simulator only approximated single pixel

updates with rapid updates of full display lines; visual quality may change when true single-pixel updates are achieved.

We built a fully interactive prototype of our frameless, integrated render-display architecture with inexpensive and widely available components, which bodes well for the architecture's commercial potential. Yet there are real rendering and display challenges that must be overcome before that potential can be realized.

In rendering, ray tracing is now widely adopted, but is currently used largely in a hybrid configuration with rasterization used for primary (eye-to-object) rays, and ray tracing only for secondary (object-to-object) shading rays. Fully ray traced games exist (Quake II [14], Portal [15], Minecraft with RTX [16], etc.) but they are largely demonstration rather than commercial projects. Recently the rate of hybrid ray-traced game releases has declined [3], with industry citing ray tracing's reduced interactivity as one cause. Paradoxically, fully ray-traced games may help realize new levels of interactivity. Moreover, ray tracing is not the only type of image-order rendering; our frameless architecture can also be realized with simpler renderers in less complex but still practical settings, like the 2D game Tetris in our prototype.

As to displays, realizing random-access and full addressability is another significant challenge. To our knowledge, no such LCD or LED panels exist. Realizing such displays will require overcoming not only a hardware but also a bandwidth challenge: in addition to color, each pixel will have to describe its location. Display technologies that do not require regular refresh may be part of the solution, enabling reduced render-display bandwidth by requiring updates only when pixel colors change. Even if fully random-access displays are not possible near-term, much of the latency reduction we describe here can be realized without them by eliminating frame buffer and synchronization delays, and feeding pixels to displays in a traditional progressive order.

In the near term, our continued work will focus on further latency reduction and evaluation. We tested progressive and stochastic scans here that visit each pixel before revisiting any. In contrast, we could lower latency even further in regions of interest (e.g., at the Tetris piece, the cursor location, or the eye's gaze point) by updating them more frequently than the rest of the image. Our Tetris game is fully interactive, enabling human experiments to assess perceived image quality and game performance. In the medium term, we will attempt to scale our prototype with larger displays, more complex rendering workloads, and more capable rendering hardware. Long term, we will prototype true random-access display architectures, possibly including always-on, state-retentive LEDs. In particular, we will focus on platforms such as smartphones, tablets, and handheld gaming consoles that already integrate rendering with display systems.

7. Acknowledgements

Our thanks to Turner Whitted, who inspired much of this work. We are also grateful to Markus Billeter for timely help with all things Raspberry Pi.

References

- [1] Shirley P, Ashikhmin M, Marschner S. Fundamentals of computer graphics. AK Peters/CRC Press; 2009.
- [2] NVIDIA CC. NVIDIA Turing GPU Architecture: Graphics reinvented. Nvidia; 2018.
- [3] Tarl. Has Ray Tracing Stalled? A 2025 State of the Tech Review; 2025. Available from: <https://gamertech.org/singlenews/Ray-Tracing>.
- [4] Spjut J, Boudaoud B, Binaee K, Kim J, Majercik A, McGuire M, et al. Latency of 30 ms benefits first person targeting tasks more than refresh rate above 60 Hz. In: SIGGRAPH Asia 2019 Technical Briefs; 2019. p. 110-3.
- [5] Jota R, Ng A, Dietz P, Wigdor D. How fast is fast enough? a study of the effects of latency in direct-touch pointing tasks. In: Proceedings of the sigchi conference on human factors in computing systems; 2013. p. 2291-300.
- [6] Ellis SR, Young MJ, Adelstein BD, Ehrlich SM. Discrimination of changes of latency during voluntary hand movement of virtual objects. In: Proceedings of the Human Factors and Ergonomics Society Annual Meeting. vol. 43. SAGE Publications Sage CA: Los Angeles, CA; 1999. p. 1182-6.
- [7] Madhusudan A, Watson B. Better frame rates or better visuals? An early report of Esports player practice in Dota 2. In: Extended Abstracts of the 2021 Annual Symposium on Computer-Human Interaction in Play; 2021. p. 174-8.
- [8] Bishop G, Fuchs H, McMillan L, Zagier EJS. Frameless rendering: Double buffering considered harmful. In: Proceedings of the 21st annual conference on Computer graphics and interactive techniques; 1994. p. 175-6.
- [9] Dayal A, Woolley C, Watson B, Luebke D. Adaptive Frameless Rendering. In: Bala K, Dutre P, editors. Eurographics Symposium on Rendering (2005). The Eurographics Association; 2005. .
- [10] Friston S, Steed A, Tilbury S, Gaydadjiev G. Construction and evaluation of an ultra low latency frameless renderer for VR. IEEE transactions on visualization and computer graphics. 2016;22(4):1377-86.
- [11] Fulmer A, Choi HJ, Watson B. Stochastic Scanning for Unsynchronized, Low-Latency Display Without Tearing. In: SID Symposium Digest of Technical Papers; 2025. p. 1885-8.
- [12] NVIDIA. NVIDIA Latency and Display Analysis Tool; 2025. Available from: <https://developer.nvidia.com/nvidia-latency-display-analysis-tool>.
- [13] Klein D, Spjut J, Boudaoud B, Kim J. Variable Frame Timing Affects Perception of Smoothness in First-Person Gaming. In: 2024 IEEE Conference on Games (CoG). IEEE; 2024. p. 1-8.
- [14] NVIDIA. QUAKE II RTX; 2019. Available from: <https://www.nvidia.com/en-us/geforce/campaigns/quake-II-rtx/>.
- [15] NVIDIA. Portal with RTX; 2025. Available from: <https://www.nvidia.com/en-us/geforce/campaigns/play-portal-with-rtx/>.
- [16] NVIDIA. Minecraft with RTX; 2020. Available from: <https://www.nvidia.com/en-us/geforce/campaigns/minecraft-with-rtx/>.